

Second Lab Assignment: Cache Simulator - Report

Francisco Fonseca - 102492; Gonalo Rua - 102604; Joo Gouveia - 102611

4.1 Directly-Mapped L1 Cache

For the first task we started by changing the way we calculate the offset (6 bits), index (8 bits) and tag (18 bits). We also changed the structure Cache so that we have multiple lines (array of size L1_BLOCK_COUNT). To access the right location inside the L1Cache array, we calculated the physical address of the corresponding block, multiplying the index with the block size. Finally, to access DRAM in case of miss with dirty flag, we had to calculate the physical address of the previous tag in that block on the DRAM array. The rest of the code remained practically unchanged.

4.2 Directly-Mapped L2 Cache

To accomplish the second task we reused the code of the Directly-Mapped L1 Cache, since the logic of the Directly-Mapped L2 Cache is almost the same, just changing the size of the index (to 9 bits since the cache size is double the size). Also we had to change the L1 Cache code for when a miss occurs it reads from the L2 Cache and when there is a miss with dirty flag we write to the L2 Cache.

4.3 2-Way L2 Cache

To add 2-way associativity to the L2 cach we only had to change the L2 cache code, while the first level cache remained unchanged. We created the struct CacheSet, which contains one array of type CacheLine, the L2 cache struct now stores one array of type CacheSet instead of CachLine. Adding associativity comes with an additional layer of complexity when it comes to identifying the desired memory address in cache, this is because while we are still provided with an index as we were before (with the direct mapped caches), this index points to a set, so we still need to search for the desired block inside it. In order to achieve this, we first set the `line` variable to `-1`. We then look through all the lines inside the set, trying to find a line that's

valid and has a tag equal to the block we're looking for. If we do find it, we set `line` to the correct value and move on to the standard cache procedures (reading or writing, same as the other 2 exercises). If we fail to find it, we had a cache miss and need to load the block into the cache before reading or writing. When this is the case, we first look for a line in the set that's not being used, as it's preferable to load the block into an invalid line as opposed to replacing a line that is valid and we might use in the future. If we fail to find one, we'll need to replace one of the blocks under use. In order to decide which one is going to be replaced, we use the LRU policy, which, given that our cache only has 2-way associativity, is fairly easy to implement. Our `CacheSet` struct has a variable that tracks the last recently used block. Since we only have two blocks, this variable is either 1 (meaning the last recently used block is the second one) or 0 (meaning that it's the first block). Whenever we access one block we just need to update the LRU value of it's set.